

Git

What is Git?

Git is *Distributed* Version Control, *open source*,

We have 3 Versions Control

1- Local : on your Device

2- Central: Like in the company, you all use server and connect Live not Local

3- Distributed: You Connect to the Internet to clone, and anytime you can make git pull and push and the reviewer can merge.

Open Source - Linus Torvalds — the same person who created the Linux kernel — is the original author of Git.

He started developing Git in **April 2005**, after the Linux kernel community lost access to the proprietary version control system they had been using, called *BitKeeper*

Why are we use Git?

In Past:

- We made Versions from files to back to the oldest one.
- We sent files to each other
- After awhile we couldn't know who and when

But with **Git** :

- Track Everything
- Unique ID
- Contribute to the same project
- Solve Conflicts
- Revert Changes

Download

On Windows:

```
winget install --id Git.Git -e --source winget
```

On Linux:

```
sudo apt update
sudo apt install git
```

Sign Up

Create Repo and clone it

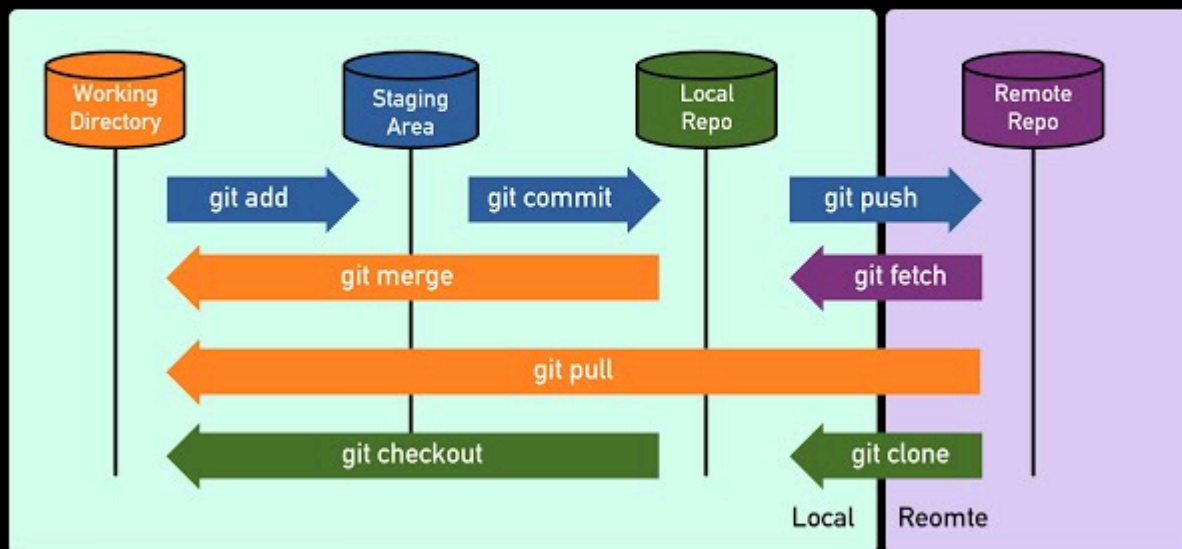
```
git clone #link
```

Create File

Types of Files

- Untracked **U**
- Tracked
- Modified **M** (orange(not in staging) and green(in staging))
- Unmodified

How Git Actually Works



To make in **Staging Area**

```
git add filename
git add . #all
git add * #all
git add *.txt #all .txt
```

To make in **Local Repo**

```
git status
git commit -m"messsage" #tell it what do you do
git commit -am"message" #add and commit
git config --global core.editor "code --wait" #you can change the editor
git commit --amend #change in the last commit message
#to write long message
git commit
```

To make in **Remote Repo**

```
git push
git push Remotename Branchname
git remote -v #to know remote
git branch #to know the branch (has *)
```

From Remote to Local

```
git pull
git pull Remotename # pull = fetch + merge
```

How to add Collaborators to your Repo

To know History

```
git log # more details
git log --oneline
git show id # to know more details about 1 commit
```

To know the difference between commits

```
git diff #Compare working directory with latest commit
git diff id1..id2 #between 2
```

Undoing

```
git restore filename # I want to cancel my modify and get the last ver. in
staged, Undo changes in working directory
git restore --staged filename #from staged to unstaged
git restore --staged * #all from staged to unstaged
git clean -n #to know what will delete from unstaged
git clean -f
```

Branching

```
git branch #to know the branch (has *) Green
git branch name #create branch
git checkout name #to enter the branch
git switch name #to enter the branch
git checkout -b name # create and enter
git branch -m rename #rename but when we in
#on master
git merge name # we can merge directly on main branch
git branch -d name # if merged done, if not will refuse that
git branch -D name # force delete
```

Stash: Save your uncommitted changes temporarily, Go back to a clean working directory, Switch branches or pull updates safely and Then restore your work later.

```
git stash # to stash files all
git stash list #see the list
git stash save "message" # stash with message
git stash pop # move from stach one by one
git stash apply #copy from stash
git stash pop 'stash@{2}' # only specified one
git stash drop #delete last one
git stash drop 'stash@{2}'
git stash show # show the contents
git stash show 'stash@{2}'
git stash clear #delete all
```

TAG: not every edit is new version so we make tags

- Light Wight: message from commit message after add files

```
git tag # to show
git tag v1.0
git push origin v1.0
```

- Annotated:

```
git tag -a v2.0 -m "message"
```

make for each tag branch because all commits will save in tag.

HEAD: head is a pointer which always points to the last commit

```
git reflog # Shows every move your `HEAD` has made
# This is extremely useful to recover "lost" commits.
git reset HEAD~1 #one step back and you can use 2 or 3 and so on,, but keeps
your working directory unchanged.
git reset --hard HEAD~1 # discards all changes in the working directory,
This permanently deletes uncommitted changes from your workspace.
#when you open git log the last commit as deleted when you step
back(disappear)
#appear in reflog not log
git reset --hard 'HEAD@{1}' # to fast forward
```

NOTE: if we have A-B-C-D and we moved to B ?

- you want to continue new line, make new commit and shouldn't go back to D
- restore D and ignore new commit `git reset --hard 'HEAD@{1}'`
- keep new B2 and D,
- make a branch before moving then move and merge

SSH Key:

is a pair of cryptographic keys used to authenticate you securely without typing a password. You get:

- Public key → You upload to GitHub/GitLab/server
- Private key → Stays on your machine (never share it)
- This allows password-less, secure communication.

Why do we use SSH keys?

- More secure than passwords
- No need to enter username/password every time
- Trusted by GitHub, GitLab, servers, VPS, etc.

How to generate an SSH key

```
ssh-keygen -t rsa -b 4096 -C "your_email@example.com"
```

when asked

```
Enter a file in which to save the key:
```

Just press **Enter** → it saves in the default location:

Add the public key to GitHub

Show your public key:

```
cat ~/.ssh/id_rsa.pub
```

Copy it and paste here:

GitHub → **Settings** → **SSH and GPG Keys** → **New SSH key**

Test the connection

```
ssh -T git@github.com
```

If successful, you'll see:

```
Hi username! You've successfully authenticated.
```